

Texel density

How many texture pixels land on one metre of model – and how to make that one number true everywhere in a scene.

ONE-PAGE REFERENCE
FOR PIXEL ART ON 3D

THE WHOLE THING

$\text{px/unit} = \sqrt{\text{UV area} \times S^2 \div \text{world area}}$
S is the texture's longest edge in pixels; world area is in m^2 , measured after object scale.

THE FORM YOU WILL ACTUALLY USE

$\text{px/unit} = S \div M$
...for a square face that uses the whole $S \times S$ texture and spans M metres. A 64 px texture across 2 m is 32 px/unit.

1 Pick one density for the whole game, and make it a power of two

PX/UNIT	1.8 M CHARACTER	1 M FLOOR TILE	0.5 M PROP
8	14 px tall	8 × 8 px	4 px
16	29 px tall	16 × 16 px	8 px
32	58 px tall	32 × 32 px	16 px
64	115 px tall	64 × 64 px	32 px
128	230 px tall	128 × 128 px	64 px

Why a power of two, and not a round character height.

You can have one or the other, never both. At 32 px/unit a 0.5 m prop gets exactly 16 px and its edges land on texel corners. Solve instead for a character exactly 32 px tall and the density becomes 17.8 px/unit – that same prop now gets 8.9 px, and every object in the scene sits half a texel off the grid.

Density is inherited by everything else in the level the character is one object. Round the number that propagates.

2 Size every texture from that one number

LONGEST EDGE	8 px/unit		16 px/unit		32 px/unit		64 px/unit	
	needs	use	needs	use	needs	use	needs	use
Small crate 0.4 m	3	4	6	8	13	16	26	32
Barrel 0.8 m	6	8	13	16	26	32	51	64
Floor tile 1 m	8	8	16	16	32	32	64	64
Character 1.8 m	14	16	29	32	58	64	115	128
Door 2.1 m	17	32	34	64	67	128	134	256
Wall section 3.2 m	26	32	51	64	102	128	205	256
Tree 6 m	48	64	96	128	192	256	384	512
Building facade 12 m	96	128	192	256	384	512	768	1024

needs = density × metres, the exact pixel count. **use** = the next power of two, because that is what the GPU wants and what keeps mipmaps honest. Round up: a texture larger than needed wastes memory, a texture smaller than needed is the blur you were trying to avoid.

3 Expect it to drift, because the default unwrap guarantees it

Unwrap a cube in Blender and you get the same UV layout whatever size the cube is. UV area is therefore constant while world area is not, so density falls as $1 \div \text{size}$ – and the spread across a scene comes out as exactly the size ratio of its objects.

A 3.2 m wall beside a 0.38 m crate is a ratio of 8.4×. Measured on that scene across 18 faces on one 32 px texture, Blender reported:

10.5 px/unit avg (2.5-21.1, 8.4x spread)
...and after correcting every face:
10.5 px/unit avg (10.5-10.5, 1.0x spread)

This is why one wall in a corridor reads crisp and the crate in front of it reads mushy, on the same texture, at the same distance.

4 Four things that quietly make the number a lie

Unapplied object scale. Density is measured in world space. A cube scaled 2× in the viewport is twice the size the mesh data thinks it is. Ctrl+A -> Scale before you measure anything.

Linear filtering. Perfect density with a smoothed texture still looks like mud. Set the image node's interpolation to **Closest** – the one people check last and should check first.

Non-uniform scale. Scale x but not y and the face has two different densities at once. No single number describes it; fix the scale rather than the UVs.

Mixed texture sizes. Two objects can share a density without sharing a texture. The density is what must match across the scene; the texture size is per object, and step 2 is how you pick it.

THE CORRECTION

A face of area A m^2 needs a UV island whose side covers this fraction of the texture:

$$\text{UV side} = D \times \sqrt{A} \div S$$

WORKED

A 2×1.5 m panel (3 m^2) at 32 px/unit on a 64 px texture:
 $32 \times \sqrt{3} \div 64 = 0.866$
so the island spans 87% of UV space, about 55 px across. Scale the island to that and the face measures back at exactly 32.

Over 1.0 means the face needs more pixels than the texture has – tile it, or move up a texture size.

Every figure here is computed by running core/uvmap.py out of the shipped texel-0.2.0.zip – the same arithmetic the add-on runs – not typed in. The generator is in the download.

Texel measures and applies this for you
z3er1n.itch.io/texel